Отчёт по лабораторной работе №13

Программирование в командном процессоре ОС UNIX. Ветвления и циклы

Хрисанова Ксения Олеговна

Содержание

Цель **5**

Задачи **6**

Теоретическое введение **7**

Выполнение лабораторной работы **8**

Контрольные вопросы **13**

Список иллюстраций

1 Написание кода 8
2 запуск скрипта 9
3 Написание кода 9
4 Написание кода 10
5 запуск скрипта 10
6 Написание кода 11
7 создание и удаление файлов 11
8 Написание кода 12
9 запуск скрипта 12

Список таблиц

Цель

Изучить основы программирования в оболочке ОС UNIX. Научится писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

Задачи

Используя команды getopts grep, написать командный файл, который анализирует командную строку с ключами: – -iinputfile — прочитать данные из указанного файла; – -ooutputfile — вывести данные в указанный файл; – -pшаблон — указать шаблон для поиска; – -C — различать большие и малые буквы; – -n — выдавать номера строк. а затем ищет в указанном файле нужные строки, определяемые ключом -p. Написать на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. Затем программа завершается с помощью функции exit(n), передавая информацию в о коде завершения в оболочку. Команд- ный файл должен вызывать эту программу и, проанализировав с помощью команды $?, выдать сообщение о том, какое число было введено. Написать командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до (например 1.tmp, 2.tmp, 3.tmp,4.tmp и т.д.). Число файлов, которые необходимо создать, передаётся в аргументы командной строки. Этот же ко- мандный файл должен уметь удалять все созданные им файлы (если они существуют). Написать командный файл, который с помощью команды tar запаковывает в архив все файлы в указанной директории. Модифицировать его так, чтобы запаковывались только те файлы, которые были изменены менее недели тому назад (использовать команду find).

Теоретическое введение

Bash (Bourne Again Shell) — это мощная командная оболочка Unix, которая используется для выполнения различных задач в терминале. Bash предоставляет интерактивный интерфейс, в котором пользователи могут вводить команды, а затем получать результаты. Она также поддерживает скрипты оболочки, которые представляют собой текстовые файлы, содержащие последовательность команд Bash для автоматизации задач. Bash широко используется в средах Unix и Linux, а также поддерживается Windows с помощью подсистемы Windows для Linux (WSL). Перечислим основные возможности этой оболочки. Обработка команд. Bash может обрабатывать как простые, так и сложные команды. Простые состоят из одного действия и, возможно, некоторых аргументов. Сложные команды могут содержать несколько простых, объединенных с помощью операторов конвейера (|), перенаправления ввода и вывода (<, >, »), условных операторов (if/else, case/esac, while/do). О них мы расскажем позже. Расширенный ввод, редактирование строк. Оболочка предоставляет функции расширенного ввода, такие как автодополнение, которое предлагает возможные варианты завершения команд и имен файлов по мере их ввода. Она также поддерживает историю, позволяя пользователям просматривать ранее введенные команды, а затем повторно их использовать. Возможность создания и запуска скриптов. Скрипты оболочки — это текстовые файлы, содержащие последовательность команд. Их можно создавать с помощью текстового редактора, а затем запускать в терминале, что дает возможность пользователям автоматизировать задачи и управлять системой. Скрипты могут содержать условные операторы, циклы, функции для обеспечения дополнительной гибкости и контроля.

Выполнение лабораторной работы

Пишу скрипт с getopts для анализа ключей командной строки(рис. @fig:001)

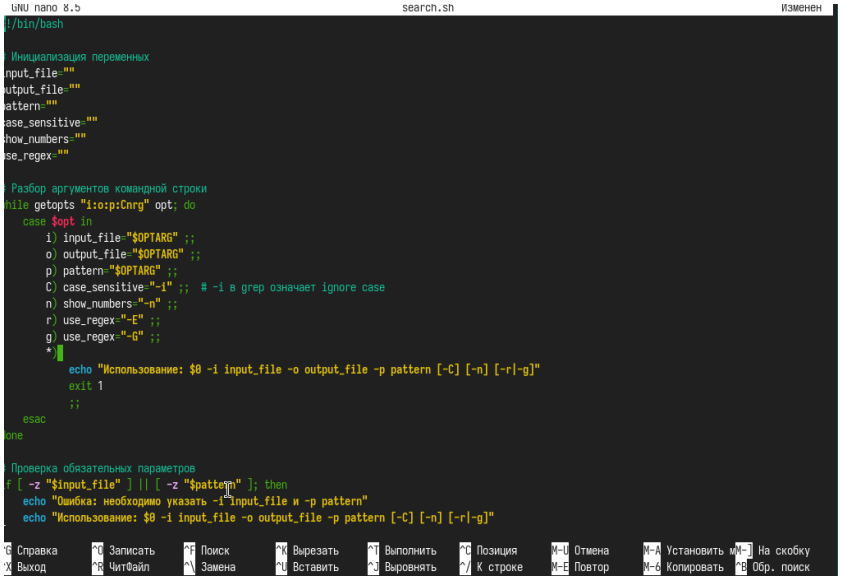

Рис. 1: Написание кода

Делаю файл исполняемым,создаю тестовый файл для проверки,тестирую поиск с разными ключами(рис. @fig:002)

```
kohrisanova@fedora:~/work/os/lab13$ ./search.sh -i test.txt -p hello -C -n
1:hello linux
kohrisanova@fedora:~/work/os/lab13$ ./search.sh -i test.txt -p Hello -r  -o output.txt
Результат сохранён в output.txt
kohrisanova@fedora:~/work/os/lab13$ cat output.txt
kohrisanova@fedora:~/work/os/lab13$ cat > test.txt << EOF
> Hello World
> hello  world
> Hello Linux
> HELLO
> EOF
kohrisanova@fedora:~/work/os/lab13$ cat test.txt
Hello World
hello  world
Hello Linux
HELLO
kohrisanova@fedora:~/work/os/lab13$ ./search.sh -i test.txt -p Hello -C
Hello World
hello  world
Hello Linux
HELLO
kohrisanova@fedora:~/work/os/lab13$  ./search.sh -i test.txt -p Hello -n
1:Hello World
3:Hello Linux
kohrisanova@fedora:~/work/os/lab13$  ./search.sh -i test.txt -p hello -C -n
1:Hello World
2:hello  world
3:Hello Linux
4:HELLO
kohrisanova@fedora:~/work/os/lab13$ ./search.sh -i test.txt -p Hello -r  -o output.txt
Результат сохранён в output.txt
kohrisanova@fedora:~/work/os/lab13$ cat output.txt
Hello World
Hello Linux
kohrisanova@fedora:~/work/os/lab13$
```

Рис. 2: запуск скрипта

Пишу программу на Си, которая определяет знак числа и возвращает код завершения(рис. @fig:003)

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int num;

    printf("Введите число: ");
    scanf("%d", &num);

    if (num > 0) {
        printf("Число %d больше нуля\n", num);
        exit(2);
    } else if (num < 0) {
        printf("Число %d меньше нуля\n", num);
        exit(1);
    } else {
        printf("Число равно нулю\n");
        exit(0);
    }
}
```

Рис. 3: Написание кода

Пишу скрипт: запускаю программу, затем через case анализирую код возврата и вывожу сообщение.(рис. @fig:004)

```
#!/bin/bash

# Запуск программы на Си
./checknum

# Анализ кода завершения
case $? in
    0) echo "Было введено число 0" ;;
    1) echo "Было введено отрицательное число" ;;
    2) echo "Было введено положительное число" ;;
    *) echo "Неизвестный код завершения" ;;
esac
```

Рис. 4: Написание кода

Запускаю и тестирую с разными числами,ввожу положительное число,потом отрицательное,а потом 0(рис. @fig:005)

```
kohrisanova@fedora:~/work/os/lab13$ nano checknum.c
kohrisanova@fedora:~/work/os/lab13$ gcc checknum.c -o checknum
kohrisanova@fedora:~/work/os/lab13$ nano run_check.sh
kohrisanova@fedora:~/work/os/lab13$ chmod +x run_chek.sh
chmod: невозможно получить доступ к 'run_chek.sh': Нет такого файла или каталога
kohrisanova@fedora:~/work/os/lab13$ chmod +x run_check.sh
kohrisanova@fedora:~/work/os/lab13$ ./run_check.sh
Введите число: 10
Число 10 больше нуля
Было введено положительное число
kohrisanova@fedora:~/work/os/lab13$ ./run_check.sh
Введите число: -5
Число -5 меньше нуля
Было введено отрицательное число
kohrisanova@fedora:~/work/os/lab13$ ./run_check.sh
Введите число: 0
Число равно нулю
Было введено число 0
kohrisanova@fedora:~/work/os/lab13$
```

Рис. 5: запуск скрипта

Пишу скрипт для создания и удаления нумерованных файлов(рис. @fig:006)

```
  GNU nano 8.5                    manage_files.sh                    Изменён
#!/bSSin/bash

# Проверка аргументов
if [ $# -ne 2 ]; then
    echo "Использование: $0 <create|delete> <N>"
    echo "  create N - создать N файлов (1.tmp, 2.tmp, ...)"
    echo "  delete N - удалить файлы (1.tmp, 2.tmp, ...)"
    exit 1
fi

action="$1"
N="$2"

# Проверка, что N - число
if ! [[ "$N" =~ ^[0-9]+$ ]]; then
    echo "Ошибка: N должно быть числом"
    exit 1
fi

case "$action" in
    create)
        for i in $(seq 1 $N); do
            touch "$i.tmp"
            echo "Создан файл: $i.tmp"
        done
        echo "Создано $N файлов"
        ;;
    delete)

^G Справка    ^O Записать   ^F Поиск      ^K Вырезать   ^T Выполнить  ^C Позиция    M-U Отмена     M-A Установить метку
^X Выход      ^R ЧитФайл    ^\ Замена     ^U Вставить   ^J Выровнять  ^/ К строке   M-E Повтор     M-6 Копировать
```

Рис. 6: Написание кода

Создаю 5 файлов,удаляю все файлы(рис. @fig:007)

```
kohrisanova@fedora:~/work/os/lab13$ nano manage_files.sh
kohrisanova@fedora:~/work/os/lab13$ chmod +x manage_files.sh
kohrisanova@fedora:~/work/os/lab13$ ./manage_files.sh create 5
bash: ./manage_files.sh: /bSSin/bash: bad interpreter: Нет такого файла или каталога
kohrisanova@fedora:~/work/os/lab13$ nano manage_files.sh
kohrisanova@fedora:~/work/os/lab13$ ./manage_files.sh create 5
Создан файл: 1.tmp
Создан файл: 2.tmp
Создан файл: 3.tmp
Создан файл: 4.tmp
Создан файл: 5.tmp
Создано 5 файлов
kohrisanova@fedora:~/work/os/lab13$ ls -la *.tmp
-rw-r--r--. 1 kohrisanova kohrisanova 0 мая  9 11:34 1.tmp
-rw-r--r--. 1 kohrisanova kohrisanova 0 мая  9 11:34 2.tmp
-rw-r--r--. 1 kohrisanova kohrisanova 0 мая  9 11:34 3.tmp
-rw-r--r--. 1 kohrisanova kohrisanova 0 мая  9 11:34 4.tmp
-rw-r--r--. 1 kohrisanova kohrisanova 0 мая  9 11:34 5.tmp
kohrisanova@fedora:~/work/os/lab13$ ./manage_files.sh delete 3
Удалён файл: 1.tmp
Удалён файл: 2.tmp
Удалён файл: 3.tmp
Удаление завершено
kohrisanova@fedora:~/work/os/lab13$ ls -la *.tmp
-rw-r--r--. 1 kohrisanova kohrisanova 0 мая  9 11:34 4.tmp
-rw-r--r--. 1 kohrisanova kohrisanova 0 мая  9 11:34 5.tmp
kohrisanova@fedora:~/work/os/lab13$ ./manage_files.sh delete 2
Файл 1.tmp не существует
Файл 2.tmp не существует
Удаление завершено
kohrisanova@fedora:~/work/os/lab13$ ls -la *.tmp
-rw-r--r--. 1 kohrisanova kohrisanova 0 мая  9 11:34 4.tmp
-rw-r--r--. 1 kohrisanova kohrisanova 0 мая  9 11:34 5.tmp
kohrisanova@fedora:~/work/os/lab13$  ./manage_files.sh delete 5
Файл 1.tmp не существует
Файл 2.tmp не существует
Файл 3.tmp не существует
Удалён файл: 4.tmp
Удалён файл: 5.tmp
Удаление завершено
```

Рис. 7: создание и удаление файлов

Пишу скрипт для архивации файлов, изменённых за последние N дней(рис. @fig:008)

```
  GNU nano 8.5                    bqckup_recent.sh                    Изменён
#!/bin/bash

# Проверка аргументов
if [ $# -lt 1 ] || [ $# -gt 2 ]; then
    echo "Использование: $0 <директория> [дней]"
    echo "  директория - каталог для архивации"
    echo "  дней - архивировать файлы изменённые за последние N дней (по умолчанию 7)"
    exit 1
fi

DIR="$1"
DAYS="${2:-7}"

# Проверка существования директории
if [ ! -d "$DIR" ]; then
    echo "Ошибка: директория $DIR не существует"
    exit 1
fi

# Имя архива с датой
ARCHIVE_NAME="backup_$(basename "$DIR")_$(date +%Y%m%d_%H%M%S).tar.gz"

# Поиск файлов, изменённых за последние DAYS дней
echo "Поиск файлов в $DIR, изменённых за последние $DAYS дней..."
FILES=$(find "$DIR" -type f -mtime -"$DAYS" -not -path "*/\.*")

if [ -z "$FILES" ]; then
    echo "Нет файлов, изменённых за последние $DAYS дней"

^G Справка   ^O Записать  ^F Поиск     ^K Вырезать  ^T Выполнить  ^C Позиция    M-U Отмена
^X Выход     ^R ЧитФайл   ^\ Замена    ^U Вставить  ^J Выровнять  ^/ К строке   M-E Повтор
```

Рис. 8: Написание кода

Создаю тестовые файлы разной давности,запускаю архивацию файлов за последние 7 дней(рис. @fig:009)

```
kohrisanova@fedora:~/work/os/lab13$ nano bqckup_recent.sh
kohrisanova@fedora:~/work/os/lab13$ chmod +x bqckup_recent.sh
kohrisanova@fedora:~/work/os/lab13$ touch old_file.txt
kohrisanova@fedora:~/work/os/lab13$ touch -d "10 days ago" very_old.txt
kohrisanova@fedora:~/work/os/lab13$ echo "test" > new_files.txt
kohrisanova@fedora:~/work/os/lab13$ ./bqckup_rexent.sh . 7
bash: ./bqckup_rexent.sh: Нет такого файла или каталога
kohrisanova@fedora:~/work/os/lab13$  ./bqckup_recent.sh . 7
Поиск файлов в ., изменённых за последние 7 дней...
Архивирую найденные файлы...
Архив создан: backup_._20260509_114118.tar.gz
Размер архива: 8,0K
kohrisanova@fedora:~/work/os/lab13$ ls -la bqckup_*.tar.gz
ls: невозможно получить доступ к 'bqckup_*.tar.gz': Нет такого файла или каталога
kohrisanova@fedora:~/work/os/lab13$ ls -la bqckup_.*.tar.gz
ls: невозможно получить доступ к 'bqckup_.*.tar.gz': Нет такого файла или каталога
kohrisanova@fedora:~/work/os/lab13$ ls -la backup_._20260509_114118.tar.gz
-rw-r--r--. 1 kohrisanova kohrisanova 4879 мая  9 11:41 backup_._20260509_114118.tar.gz
kohrisanova@fedora:~/work/os/lab13$ tar -tzf backup_._20260509_114118.tar.gz
./search.sh
./rest.txt
./test.txt
./output.txt
./checknum.c
./checknum
./run_check.sh
./manage_files.sh
./bqckup_recent.sh
./old_file.txt
./new_files.txt
kohrisanova@fedora:~/work/os/lab13$
```

Рис. 9: запуск скрипта

Контрольные вопросы

1. Каково предназначение команды getopts? Осуществляет синтаксический анализ командной строки, выделяя флаги, и используется для объявления переменных. Синтаксис команды следующий: getopts option-string variable. Флаги – это опции командной строки, обычно помеченные знаком минус; Например, -F является флагом для команды ls -F. Иногда эти флаги имеют аргументы, связанные с ними. Программы интерпретируют эти флаги, соответствующим образом изменяя свое поведение. Строка опций option-string — это список возможных букв и чисел соответствующего флага. Если ожидается, что некоторый флаг будет сопровождаться некоторым аргументом, то за этой буквой должно следовать двоеточие. Соответствующей переменной присваивается буква данной опции. Если команда getopts может распознать аргумент, она возвращает истину. Принято включать getopts в цикл while и анализировать введенные данные с помощью оператора case. Предположим, необходимо распознать командную строку следующего формата: testprog -ifile_in.txt -ofile_out.doc -L -t -r Вот как выглядит использование оператора getopts в этом случае: while getopts o:i:Ltr optletter do case $optletterino)oflag = 1; oval$ =OPTARG;; i) iflag=1; ival=$OPTARG;; L) Lflag=1;; t) tflag=1;; r) rflag=1;; *) echo Illegal option $optletter esac done Функция getopts включает две специальные переменные среды – OPTARG и OPTIND. Если ожидается дополнительное значение, то OPTARG устанавливается в значение этого аргумента (будет равна file_in.txt для опции i и file_out.doc для опции o) . OPTIND является числовым индексом на упомянутый аргумент. Функция getopts также понимает переменные типа массив, следовательно, можно использовать ее

в функции не только для синтаксического анализа аргументов функций, но и для анализа введенных пользователем данных.

2. Какое отношение метасимволы имеют к генерации имён файлов? При перечислении имён файлов текущего каталога можно использовать следующие символы: − соответствует произвольной, в том числе и пустой строке; ? − соответствует любому одинарному символу; [c1-c2] − соответствует любому символу, лексикографически находящемуся между символами c1 и c2. Например, echo * − выведет имена всех файлов текущего каталога, что представляет собой простейший аналог команды ls; ls .c − выведет все файлы с последними двумя символами, совпадающими с .c. echo prog.? − выведет все файлы, состоящие из пяти или шести символов, первыми пятью символами которых являются prog.. [a-z] − соответствует произвольному имени файла в текущем каталоге, начинающемуся с любой строчной буквы латинского алфавита.

3. Какие операторы управления действиями вы знаете? Часто бывает необходимо обеспечить проведение каких-либо действий циклически и управление дальнейшими действиями в зависимости отрезультатов проверки некоторого условия. Для решения подобных задач язык программирования bash предоставляет возможность использовать такие управляющие конструкции, как for, case, if и while. С точки зрения командного процессора эти управляющие конструкции являются обычными командами и могут использоваться как при создании командных файлов, так и при работе в интерактивном режиме. Команды, реализующие подобные конструкции, по сути, являются операторами языка программирования bash. Поэтому при описании языка программирования bash термин оператор будет использоваться наравне с термином команда. Команды ОС UNIX возвращают код завершения, значение которого может быть использовано для принятия решения о дальнейших действиях. Команда test, например, создана специально для использования в командных файлах. Единственная функция этой команды заключается в выработке кода завершения.

4. Какие операторы используются для прерывания цикла? Два несложных способа позволяют вам прерывать циклы в оболочке bash. Команда break завершает выполнение цикла, а команда continue завершает данную итерацию блока операторов. Команда break полезна для завершения цикла while в ситуациях, когда условие перестаёт быть правильным. Команда continue используется в ситуациях, когда больше нет необходимости выполнять блок операторов, но вы можете захотеть продолжить проверять данный блок на других условных выражениях.

5. Для чего нужны команды false и true? Следующие две команды ОС UNIX используются только совместно с управляющими конструкциями языка программирования bash: это команда true, которая всегда возвращает код завершения, равный нулю (т.е. истина), и команда false, которая всегда возвращает код завершения, не равный нулю (т. е. ложь).

6. Что означает строка if test -f man$s/$i.$s, ?$iftest - fman$s/$i.s проверяет, существует ли файл man$s/$i.$s и является ли этот файл обычным файлом. Если данный файл является каталогом, то команда вернет нулевое значение (ложь).

7. Объясните различия между конструкциями while и until. Выполнение оператора цикла while сводится к тому, что сначала выполняется последовательность команд (операторов), которую задаёт список-команд в строке, содержащей служебное слово while, а затем, если последняя выполненная команда из этой последовательности команд возвращает нулевой код завершения (истина), выполняется последовательность команд (операторов), которую задаёт список-команд в строке, содержащей служебное слово do, после чего осуществляется безусловный переход на начало оператора цикла while. Выход из цикла будет осуществлён тогда, когда последняя выполненная команда из последовательности команд (операторов), которую задаёт список-команд в строке, содержащей служебное слово while, возвратит ненулевой код завершения (ложь). При замене в операторе цикла while служебного слова while на until условие, при выполнении

которого осуществляется выход из цикла, меняется на противоположное. В остальном оператор цикла while и оператор цикла until идентичны.